
```
AttributeError: 'PrivatePerson' object has no attribute '__privateData'
```

Similarly, if you create a method name that starts with two underscores, any attempt by client software to call the method will generate an error.

Python provides this ability by performing *name mangling*. Behind the scenes, Python changes any name that starts with two underscores by prepending it with an underscore and the name of the class, so `__<name>` becomes `_<className>__<name>`. For example, in the `PrivatePerson` class, Python will change `self.__privateData` to `self._PrivatePerson__privateData`. Therefore, if a client tries to use the name `oPrivatePerson.__privateData`, that name won't be recognized.

This is a subtle change designed as a deterrent to using direct access, but you should note that it doesn't absolutely guarantee privacy. If the client programmer knows how this works, they can still access the instance variable with `<object>._<className>__<name>` (or, in our example, `oPrivatePerson._PrivatePerson__privateData`).

Decorators and @property

At a high level, a decorator is a method that takes another method as an argument and extends the way the original method works. (Decorators can also be functions that decorate functions or methods, but I'll concentrate on methods.) Decorators are an advanced topic and are generally beyond the scope of this book. However, there is a set of built-in decorators that provide a compromise between direct access and the use of getters and setters in a class.

A decorator is written as a line that starts with the `@` symbol followed by a decorator name and is placed directly before the `def` statement of a method. This applies the decorator to a method to add to its behavior:

```
@<decorator>  
def <someMethod>(self, <parameters>)
```

We'll use two built-in decorators and apply them to two methods in a class to implement a *property*.